

浙江大学

本科实验报告

课程名称：计算机组成与设计

姓名：张序鸿

学院：竺可桢学院

专业：计算机科学与技术

学号：3230103265

指导教师：杨莹春

2024 年 9 月 28 日

浙江大学实验报告

课程名称： 计算机组成与设计 实验类型： 综合

实验项目名称： ALU、Regfiles 设计、有限状态机

学生姓名： 张序鸿 专业： 混合班 学号： 3230103265

指导老师： 杨莹春、潘之杰 助教： 张立冬、马致远

实验地点： 东 4-509 实验日期： 2024 年 9 月 18 日

一、实验目的和要求

1. 复习寄存器传输控制技术
2. 掌握 CPU 的核心组成：数据通路与控制器
3. 设计数据通路的功能部件
4. 进一步了解计算机系统的基本结构
5. 熟练掌握 IP 核的使用方法
6. 复习有限状态机的基本概念
7. 掌握有限状态机的两种模型
8. 设计有限状态机解决实际问题

二、实验内容和原理

内容：

任务一：设计实现数据通路部件 ALU

任务二：设计实现数据通路部件 Register Files

---采用硬件描述语言的设计方法

任务三：设计有限状态机完成序列检测器并测试

---用状态机设计序列检测器

原理：

2.1 ALU 基本运算及功能

ALU 作为数据通路的功能部件之一，要实现 5 个基本运算以及 3 个拓展运算。同时在得出结果的同时，进行 zero（overflow 未要求）的判断。

ALU Control Lines	Function	note
000	And	兼容
001	Or	兼容
010	Add	兼容
110	Sub	兼容
111	Set on less than	
100	nor	扩展
101	srl	扩展
011	xor	扩展

图 1. ALU 基本运算及对应 control

此外，本实验采用两种方式实现 ALU，结构化描述 与 功能性描述设计。

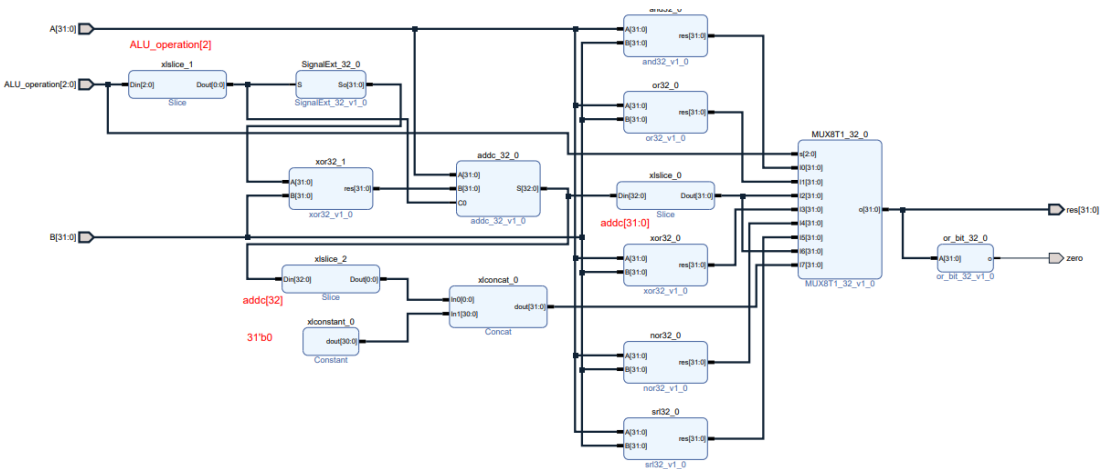


图 2. ALU 逻辑原理图（结构化描述）

2.2 数据通路部件 Register Files

本实验还需实现并优化数字系统的功能部件之一——Register files，并主要进行行为描述，最后进行仿真。

端口要求：

- 二个读端口：Rs1_addr ; Rs1_data ; Rs2_addr ; Rs2_data 2
- 一个写端口： Wt_addr ; Wt_data
- 写信号： RegWrite

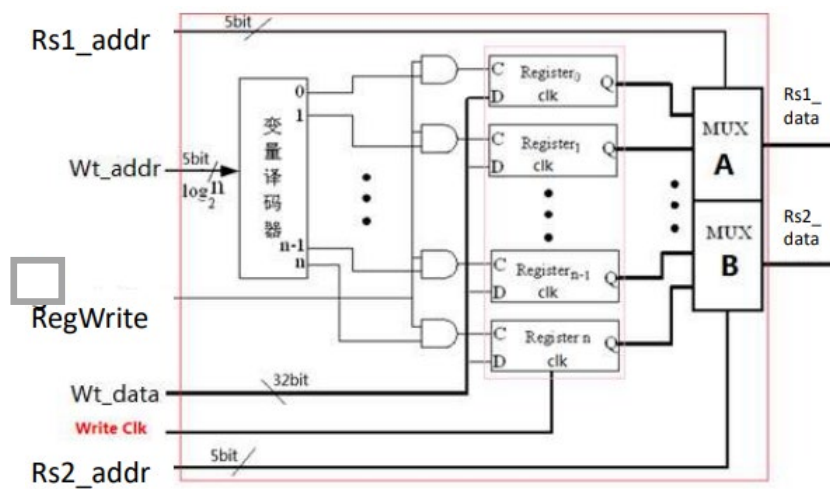


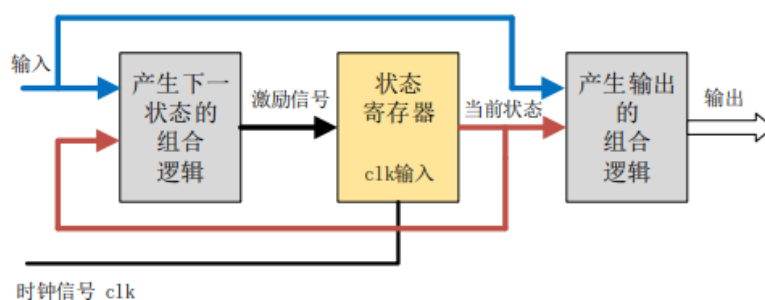
图 3. Regs 结构原理图

2.3 有限状态机的基本原理

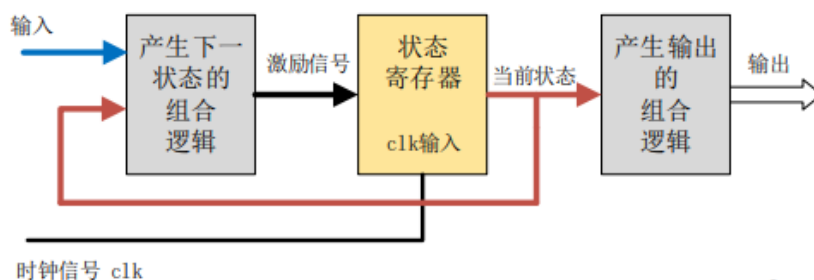
2.3.1 状态机基本概念

有限状态机（Finite State Machine，简称 FSM）在有限个状态之间按一定规律转换的时序电路。有限状态机通常是由寄存器组和组合逻辑组成时序电路，根据当前状态和输入信号可以控制下一个状态的跳转，有限状态机在电路中通常是作为控制模块，作为整个电路模块的核心而存在，其主要包括两大类：Mealy 状态机、Moore 状态机：

- Mealy 状态机的 组合逻辑输出 $\propto$ both 当前状态 + 目前输入



- Moore 状态机的 组合逻辑输出 $\propto$ 当前状态



2.3.2 状态机设计步骤

状态机设计方法多样，本实验采用三段式描述，输出信号与状态跳转分开描述，并且状态跳转用组合逻辑来控制。

• 接口定义：

接口	接口定义
clk	系统时钟
rst_n	系统复位
X	序列输入
Y	检测输出

• 状态定义 和 编码：

状态机的编码方式主要包括：二进制码、格雷码、独热码。

格雷码相对于二进制码，在状态跳转的时候，只有单比特翻转，它的功耗相对较低。

独热码相对于格雷码或者二进制码而言，它增加了两个寄存器来表示状态，但是它会更节省组合逻辑电路，因为它在比较状态的时候，只需要比较一个比特位，那么其电路的速度和可靠性就会增加。

四个状态的编码方式表

	二进制	格雷码	独热码
S0	00	00	0001
S1	01	01	0010
S2	10	11	0100
S3	11	10	1000

三、实验过程和数据记录

3.1 设计 ALU（包含结构性描述、功能性描述）

（1）Vivado 新建工程

- 利用 Vivado 新建工程，命名 OExp01 - ALU;
- 新建文件 ALU.v,将 Exp0 中导出的 Verilog 文件 and32、or32、ADC32、xor32、nor32、srl32、SignalExt_32、mux8to1_32、or_bit_32 添加入工程;
- 最后正确选择 FPGA 芯片——xc7a100tcs9364-1

（2）ALU top 文件编写:

```
`timescale 1ns / 1ps
module ALU(
    input [31:0] A,
    input [2:0] ALU_operation,
    input [31:0] B,
    output [31:0] res,
    output zero
);
    wire [31:0] and32_res;           // AND 操作结果
    wire [31:0] or32_res;           // OR 操作结果
    wire [32:0] ADC_res;            // 加法（带进位）操作结果（32 位+进位）
    wire [31:0] xor32_res_0;        // XOR 操作结果(获得 B 的取反)
    wire [31:0] xor32_res_1;        // XOR 操作结果(获得 A 与 B 的异或)
    wire [31:0] nor32_res;          // NOR 操作结果
    wire [31:0] srl32_res;          // 逻辑右移操作结果
    wire [31:0] SignalExt_32_res;
    wire [31:0] MUX8T1_32_0_o;      // MUX 的输出
    and32 and32_0(
        .A(A),
        .B(B),
        .res(and32_res)
    );
    or32 or32_0(
        .A(A),
        .B(B),
        .res(or32_res)
    );
    ADC32 ADC32_0(
        .A(A),
        .B(xor32_res_0),
        .C0(ALU_operation[2]),
        .S(ADC_res)
    );
```

```

);
xor32 xor32_0(
    .A(SignalExt_32_res),
    .B(B),
    .res(xor32_res_0)
);
xor32 xor32_1(
    .A(A),
    .B(B),
    .res(xor32_res_1)
);
nor32 nor32_0(
    .A(A),
    .B(B),
    .res(nor32_res)
);
srl32 srl32_0(
    .A(A),
    .B(B),
    .res(srl32_res)
);
SignalExt_32 SignalExt_32_0(
    .S(ALU_operation[2]),
    .So(SignalExt_32_res)
);
or_bit_32 or_bit_32_0(
    .A(MUX8T1_32_0_o),
    .o(zero)
);
MUX8T1_32 MUX8T1_32_0(
    .I0(and32_res),
    .I1(or32_res),
    .I2(ADC_res[31:0]),
    .I3(xor32_res_1),
    .I4(nor32_res),
    .I5(srl32_res),
    .I6(ADC_res[31:0]),
    .I7({31'b0, ADC_res[32]}),
    .o(MUX8T1_32_0_o),
    .s(ALU_operation)
);
assign res = MUX8T1_32_0_o;
endmodule

```

P.s. 其中调用的各个子模块（简单实现）在报告中未给出。

以上为结构性描述，以下为功能性描述：

```
`timescale 1ns / 1ps
module ALU_2(
    input [31:0] A,
    input [2:0] ALU_operation,
    input [31:0] B,
    output reg [31:0] res,
    output zero
);

    wire [32:0] addc;
    assign addc = {1'b0 , A} + {ALU_operation[2], ~B} + ALU_operation[2];
    always @ (*)
        case (ALU_operation)
            3'b000: res = A & B;
            3'b001: res = A | B;
            3'b010: res = A + B;
            3'b011: res = A ^ B;
            3'b100: res = ~ ( A | B );
            3'b101: res = A >> B[4:0];
            3'b110: res = A - B;
            3'b111: res = {31'b0, addc[32]};
        endcase
    assign zero = (res == 32'h0);
endmodule
```

(3) ALU 模块 行为仿真

ALU 工程的 Simulation Source 中添加仿真波形文件 ALU_tb.v:

```
`timescale 1ns / 1ps
module ALU_tb();
    reg [31:0] A, B;
    reg [2:0] ALU_operation;
    wire [31:0] res;
    wire zero;

    ALU ALU_uut(
        .A(A),
        .B(B),
        .ALU_operation(ALU_operation),
        .res(res),
        .zero(zero)
    );
endmodule
```



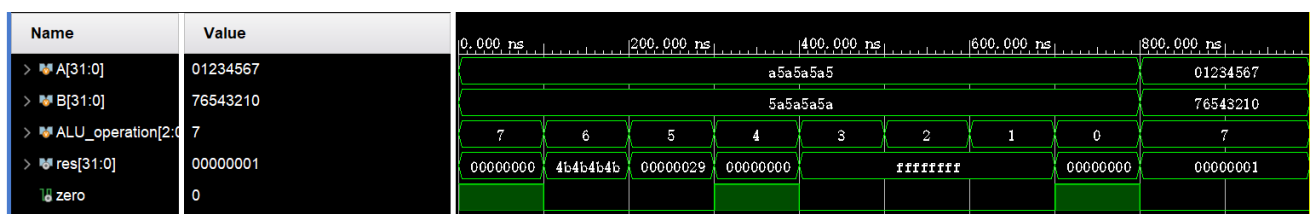
```

initial begin
    A=32'hA5A5A5A5;
    B=32'h5A5A5A5A;
    ALU_operation =3'b111;
    #100;
    ALU_operation =3'b110;
    #100;
    ALU_operation =3'b101;
    #100;
    ALU_operation =3'b100;
    #100;
    ALU_operation =3'b011;
    #100;
    ALU_operation =3'b010;
    #100;
    ALU_operation =3'b001;
    #100;
    ALU_operation =3'b000;
    #100;
    A=32'h01234567;
    B=32'h76543210;

    ALU_operation =3'b111;
end
endmodule

```

右键 sim_1 菜单选择 Run Behavioral Simulation 进行行为仿真。在 SIMULATION 窗口显示完整波形，对于两种不同描述设计的 ALU 都能得到如下图的仿真波形：



可以看到在 0 ~ 800ns 期间，ALU 的基本运算以及拓展功能能够正常实现。

如：ALU 操作数为 6 时，进行减法 $A - B = a5a5a5a5 - 5a5a5a5a = 4b4b4b4b$ ，并且 res 不为 0，因此结果 zero 也为 0，符合预期。

3.2 设计实现数据通路部件 Register Files

(1) Vivado 新建工程

- 利用 Vivado 新建工程，命名 OExp01 - Regs;
- 新建文件 Regs.v，添加到工程，采用行为描述设计；
- 最后正确选择 FPGA 芯片——xc7a100tcsg324-1

(2) Regs top 文件编写：

```
`timescale 1ns / 1ps

module Regs(
    input clk,
    input rst,
    input [4:0] Rs1_addr,
    input [4:0] Rs2_addr,
    input [4:0] Wt_addr,
    input [31:0] Wt_data,
    input RegWrite,
    output [31:0] Rs1_data,
    output [31:0] Rs2_data
);
    reg [31:0] register [1:31];
    integer i;

    assign Rs1_data = (Rs1_addr == 0) ? 0 : register[Rs1_addr];
    assign Rs2_data = (Rs2_addr == 0) ? 0 : register[Rs2_addr];

    always @(posedge clk or posedge rst) begin
        if (rst == 1)
            for (i = 1; i < 32; i = i + 1)
                register[i] <= 0;
        else if ((Wt_addr != 0) && (RegWrite == 1))
            register[Wt_addr] <= Wt_data;
        end
    end
endmodule
```

(3) Regs 模块 行为仿真

Regs 工程的 Simulation Source 中添加仿真波形文件 Regs_tb.v:

```
`timescale 1ns / 1ps
module Regs_tb();
    reg clk;
    reg rst;
    reg [4:0] Rs1_addr;
    reg [4:0] Rs2_addr;
    reg [4:0] Wt_addr;
    reg [31:0] Wt_data;
    reg RegWrite;
    wire [31:0] Rs1_data;
    wire [31:0] Rs2_data;

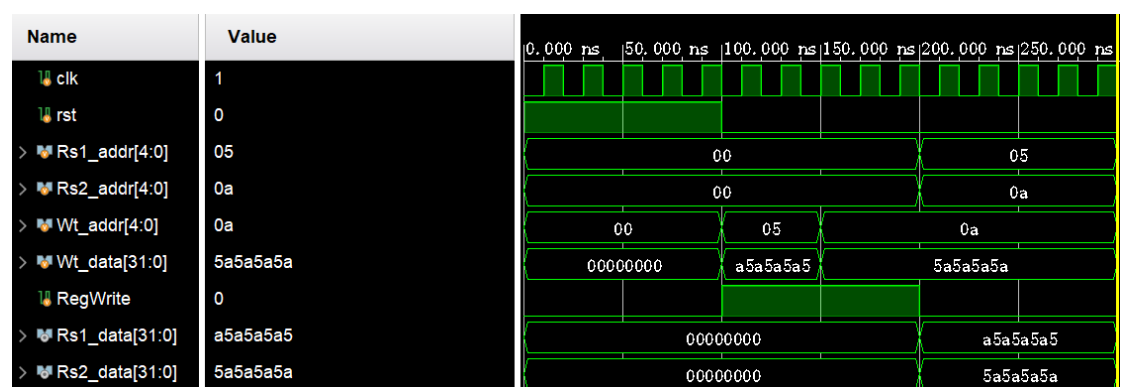
    Regs Regs_U(
        .clk(clk),
        .rst(rst),
        .Rs1_addr(Rs1_addr),
        .Rs2_addr(Rs2_addr),
        .Wt_addr(Wt_addr),
        .Wt_data(Wt_data),
        .RegWrite(RegWrite),
        .Rs1_data(Rs1_data),
        .Rs2_data(Rs2_data)
    );

    always #10 clk = ~clk;
    initial begin
        clk = 0;
        rst = 1;
        RegWrite = 0;
        Wt_data = 0;
        Wt_addr = 0;
        Rs1_addr = 0;
        Rs2_addr = 0;
        #100
        rst = 0;
        RegWrite = 1;
        Wt_addr = 5'b00101;
        Wt_data = 32'h5a5a5a5a;
        #50
        Wt_addr = 5'b01010;
        Wt_data = 32'h5a5a5a5a;
    end
endmodule
```

```
#50
RegWrite = 0;
Rs1_addr = 5'b00101;
Rs2_addr = 5'b01010;

#100 $stop();
end
endmodule
```

右键 sim_1 菜单选择 Run Behavioral Simulation 进行行为仿真。在 SIMULATION 窗口显示完整波形，得到如下图的仿真波形：



100 ~ 150ns 之间，Wt_data = a5a5a5a5 和 Wt_addr=05，写入地址 05 处；
150 ~ 200ns 之间，Wt_data = 5a5a5a5a 和 Wt_addr=0a，写入地址 0a 处；
200 ~ 250ns 之间，Rs1_addr = 05，读写地址 05，从而 Rs1_dara = a5a5a5a5；
同理，此时 Rs2_data 写入 5a5a5a5a，从仿真结果可以看出功能正确。

3.3 设计实现三段式 Moore 状态机

(1) Vivado 新建工程

- 利用 Vivado 新建工程，命名 OExp01 - seq;
- 新建文件 seq.v，添加到工程，采用行为描述设计；
- 最后正确选择 FPGA 芯片——xc7a100tcsg324-1

设计一个序列检测器，检测的序列为“1110010”；当输入信号 X 依次为“1110010”时，输出信号 Y 输出一个高电平，否则输出信号 Y 为低电平。

(2) seq top 文件编写:

采用三段式描述实现 Moore 状态机:

```
`timescale 1ns / 1ps

module seq(
    input clk,
    input reset,
    input in,
    output out
);    // 结构定义

    parameter [2:0]
        S0=3'b000,S1=3'b001,S2=3'b010,S3=3'b011,
        S4=3'b100,S5=3'b101,S6=3'b110,S7=3'b111;
    reg [2:0] curr_state;
    reg [2:0] next_state;    // 状态定义与编码

    always @(posedge clk or negedge reset)
    begin
        if(!reset)
            curr_state <= S0;
        else
            curr_state <= next_state;
    end    // 时钟信号控制状态跳转（时序逻辑）

    always @(curr_state or in) begin
        case(curr_state)
            S0: next_state = in ? S1 : S0;
            S1: next_state = in ? S2 : S0;
            S2: next_state = in ? S3 : S0;
            S3: next_state = in ? S3 : S4;
            S4: next_state = in ? S1 : S5;
            S5: next_state = in ? S6 : S0;
            S6: next_state = in ? S2 : S7;
            S7: next_state = in ? S1 : S0;
            default: next_state = S0;    // 此处逻辑根据“序列”设置
        endcase
    end    // 后续状态判断（组合逻辑）

    assign out = (curr_state == S7) ? 1'b1 : 1'b0;    // 结果输出（组合逻辑）
endmodule
```

(3) seq 状态机行为仿真

seq 工程的 Simulation Source 中添加仿真波形文件 seq_tb.v:

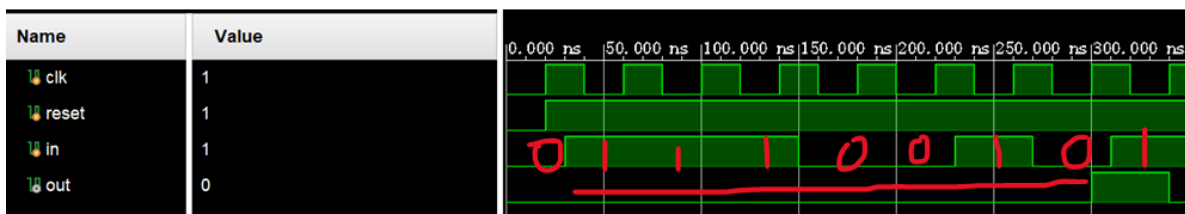
```
`timescale 1ns / 1ps
module seq_tb();
    reg clk;
    reg reset;
    reg in;
    wire out;

    seq uut(
        .clk(clk),
        .reset(reset),
        .in(in),
        .out(out)
    );
    always #20 clk = ~clk;

    initial begin
        clk = 0;
        reset = 0;
        #20 reset = 1;
    end

    initial
    begin
        in = 0;
        #30 in = 1;
        #40 in = 1;
        #40 in = 1;
        #40 in = 0;
        #40 in = 0;
        #40 in = 1;
        #40 in = 0;
        #40 in = 1;
        #40
        $finish;
    end
endmodule
```

右键 sim_1 菜单选择 Run Behavioral Simulation 进行行为仿真。在 SIMULATION 窗口显示完整波形，得到如下图的仿真波形：



当且仅当输入 in 的数据组合与所给序列“1110010”相同时，out 得出 1。

这与 Moore 状态机所要求的——当前状态决定输出 符合。

当前设计符合功能需求。

四、实验结果分析

各实验的具体仿真结果已在上文给出，因此此处只分析思考题。

• 如何给 ALU 增加溢出功能

(1) 对于有符号数加法（减法可转为加法），需要看 A、B、res 的最高位：

当两个符号不同的数相加，不会发生溢出（res 的位数 $\leq$ 任何一个数）；

当两个正数相加时，结果为负数（符号位 1），或两个负数相加时，结果为正数（符号位 0），则发生了溢出。

如果操作数 A 和 B 的符号位相同，结果 S 的符号位不同，则发生溢出。

即：溢出条件 = (A 符号位 == B 符号位) & (S 符号位 $\neq$ A 符号位)

也可以通过观察最高位的进位和次高位的进位来确定， $C_{n-1} \text{ xor } C_n$ ，1 则发生溢出，0 则未发生。

(2) 对于无符号数，书中给出了解释：

幸运的是，编译器可以轻松检查出使用分支指令的无符号溢出。如果总和小于加数中的任何一个，则加法溢出，而如果差大于被减数，则减法溢出。

- 分析逻辑实验的 **Register Files** 设计

- 本实验做了哪些优化?

本实验将 write 改为了时钟驱动，从而使得数据能够在时钟控制下迅速写入，但是 read 仍为组合逻辑。

- 逻辑 reg file 直接使用会有哪些问题?

逻辑 reg file 采用异步操作，可能在后期会出现集成到 SOC 中与 RAM 或 CPU 不同步的问题。

五、讨论与心得

这次实验总体来说还是比较简单的。

Lab1 主要就是.v 文件的添加、子模块调用以及先前一些功能部件的再实现再优化，可以算是过了一个暑假后真正的 warmup。

新课程、新老师、新助教，希望后面的课程实验能够顺利进行！